
The LLM Debate

Project 3

Tyler DeLorey
tyler.delorey1@marist.edu

December 7th, 2025

Contents

1	How to Setup Project	2
1.1	Installing Docker	2
1.2	Commands to Build Image and Run Containers	2
2	Explanations	4
2.1	Dockerfile	4
2.2	Docker Compose	5
2.3	Variables and Structures	6
2.4	Helper Functions	9
2.5	Main Program	14
3	Conclusion	19
3.1	Quick Notes	19
3.2	Use of AI	19
3.3	My Thoughts	20

1 How to Setup Project

1.1 Installing Docker

This project uses Docker in order to set up everything correctly. I would recommend installing Docker Desktop, as that is what I used and it worked out. To learn how to install Docker Desktop, click the link [here](#).

Now you should have installed Docker Desktop and created an account. You can type `docker version` in your CLI of choice to see if it installed correctly. If it did, in the directory of this project, if you aren't logged in, type `docker login` to login to your account.

I recommend using Docker Desktop, especially for this Project, because there is a part where we need to open a certain port in the model runner, and this is very easy to do through a command using Docker Desktop (as will be explained later).

1.2 Commands to Build Image and Run Containers

The first thing to make sure that is set up correctly is enabling port 12434 in the model-runner in Docker. This is used to connect the LLM to a port so that the HTTP POST requests will be successful. Without enabling 12434, it will say that the POST request connection refused. To do this using Docker Desktop (which is recommended), use the command:

- `docker desktop enable model-runner --tcp=12434`

In your directory, make sure you have the `proj3-image.tar` file installed, since that is the Docker Image. To load this Docker Image into your own Docker, use the command:

- `docker load -i proj3-image.tar`

The first time you run this program, you are going to have to download the LLM that this project uses. The default LLM provided in the `docker-compose.yml` file is around 2 GB, which is the smallest size that I could find that still gives meaningful responses (please look at the `docker-compose.yml` section to find out how to change the model if you want to). To download this, simply use the command:

- `docker-compose up -d llm`

and wait for it to finish installing! Note - Do not worry about the message that says: service "llm" has no container to start

Now that the image is loaded and the LLM is downloaded, you can run a container and the program using the following command:

- `docker-compose run --rm -it proj3`
 - `--rm` makes sure the container is removed after we run it, since for this project and my implementation, the container doesn't need to run after the program ends.
 - `-it` is actually two flags combined
 - * `-i` = interactive: Keeps STDIN open even if not attached.
 - * `-t` = pseudo-TTY: Allocates a pseudo terminal so the container behaves like a real terminal. This is used to make the debate output flow like an actual LLM would [[Using -it in docker run Command](#)]

If you want to change the debate topic, the religions of the debaters, sentence count, or turn count, you can simply change the `docker-compose.yml` file and rebuild everything using:

- `docker-compose build`
- `docker-compose run --rm -it proj3`

assuming you already have the LLM installed. If you don't have the model installed, use the command mentioned previously.

If you want to save the image on your computer, run the command:

- `docker save -o proj3_image.tar proj3-proj3:latest`

If you want to reset everything, here are the commands you can use:

- `docker-compose down`
- `docker desktop disable model-runner`

The next section has all of the needed information for the software architecture used in the code and why these decisions were made.

2 Explanations

2.1 Dockerfile

Below is my implementation of the Dockerfile, the file that is used (with the source code) to build the Docker Image for this program.

```
1 # -- BUILD STAGE --
2 FROM golang:1.25.1-alpine AS build-stage
3
4 # Install build dependencies
5 RUN apk add --no-cache upx
6
7 WORKDIR /app
8
9 # Copy the source code
10 COPY *.go .
11
12 # Build static binary with stripped debug info
13 RUN CGO_ENABLED=0 GO111MODULE=off go build -ldflags="-s -w" -o
    proj3
14
15 # Compress binary with UPX
16 RUN upx --best --lzma proj3
17
18 # -- RUNTIME STAGE --
19 FROM scratch
20
21 WORKDIR /app
22
23 # Copy binary from builder stage
24 COPY --from=build-stage /app/proj3 .
25
26 # Run default command
27 CMD ["/proj3"]
```

Figure 2.1: Dockerfile

The build stage is where a chunk of the code and the data is. This data is only loaded once, as Go modules are cached to speed up rebuilds. It uses the Go Alpine Image, which is small, lightweight, and efficient for our program. It installs build dependencies, like `upx`, which is a small image for usage in multi-stage Docker builds to compress binary files like Go or Rust [UPX Source17]. We set our working directory to `/app` and we get our source code into the Image. Note that this project does not need a `go.mod` or a `go.sum` file since there are no external dependencies. After we get our source code into the Image, it will build the static binary (with some other extra settings to lower the size of the Image), and it will be compressed using `UPX`.

At the runtime stage, we start from an empty Image. This greatly reduces the Image size because the runtime build only contains what we explicitly copy. All

that is done at this stage is copying the compressed static binary from the build stage and then specify the default command that will run when a container starts from this Docker Image.

The Docker Image size is only around 3.8 MB, mostly because the runtime uses scratch, but also because we are using a multi-stage build and requiring containing only the compiled binary and things we definitely need.

Most of the information I used to create my Dockerfile, for Golang specifically, was found here: [\[Golang Build Images Guide24\]](#).

2.2 Docker Compose

Here I will explain the fundamentals of the `docker-compose.yml`, including the two services in it. The first service I will be talking about is the `llm` service, as shown below:

```
17 llm:
18   provider:
19     type: model
20     options:
21       model: ai/llama3.2:latest
```

Figure 2.2: LLM Service in docker-compose.yml

When using the command `docker-compose up -d llm`, it will run this LLM service. Using this format, it will automatically download this model to Docker [\[Setting Up LLM Within Docker\]](#). For this project, the default model provided (that is perfect for this project) is `ai/llama3.2:latest`, which is noted on Line 21 (this model is also mentioned in the `proj3` service). This service only needs to be ran once. When the LLM is downloaded, it does not need to be downloaded again. The second service I'll explain is the main `proj3` service, as shown below:

```
2  proj3:
3    build: .
4    container_name: proj3
5    environment:
6      - BASE_URL=http://host.docker.internal:12434/engines/llama.
7        cpp/v1/
8        - MODEL=ai/llama3.2:latest
9
10     - LLM_ONE_RELIGION=Catholic
11     - LLM_TWO_RELIGION=Atheist
12     - TOPIC=Abortion
13     - TURNS=3
14     - SENTENCES=3
15    volumes:
16      - ./:/app/output
```

Figure 2.3: Proj3 Service in docker-compose.yml

This service first defines a `BASE_URL` pointing to a locally hosted LLM API (a LLAMA-based engine), and a `MODEL` to specify which AI model to use (which is the same as the model displayed in the llm service) [[Setting Up LLM Within Docker](#)]. Additional variables like `LLM.ONE_RELIGION` and `LLM.TWO_RELIGION` configure the simulated perspectives of two language models (defaulted to Catholic and Atheist), while `TOPIC` sets the discussion subject (in this case, abortion). The `URNS` and `SENTENCES` variables control how many dialogue exchanges occur and how long each response should be. Finally, the `volumes` section mounts the current project directory into the container at `/app/output`, allowing the container to write files directly to the host machine.

The reason why I set it up like this is because it lowers the size of the Docker Image. Since the Docker Model is completely separate, it will not interfere with the Docker Image at all. Also, switching out models is theoretically super easy, as long as it is LLAMA-based and produces JSON output when ran. Of course, you still have to download the model (which is less than 1.8 GB), but it remains separate, and you only have to download it once on your machine.

I chose the `ai/llama3.2:latest` model because it was the smallest one I found that actually produces reasonable output. I tried to use `ai/smollm2:latest`, which is way smaller, but the output produced was not reasonable at all, so I switched to `ai/llama3.1:latest`, and then since there was a bug with a 502 error (bad gateway), I switched to version 3.2 (`ai/llama3.2:latest`), which is actually smaller than previous 3.1 version.

2.3 Variables and Structures

Before diving deep into how the program actually works, I must explain some of the variables that the program requires to run. There are some environment variables, like talked about before, but there are also some other constants to look at. Below are some global variables the program requires:

```

17 // Set global environment variables
18 var (
19     // Using the user's provided environment variable names
20     BASE_URL string = os.Getenv("BASE_URL")
21     model     string = os.Getenv("MODEL")
22
23     // Debater 1 (ID 1)
24     religion1 string = os.Getenv("LLM_ONE_RELIGION")
25     // Debater 2 (ID 2)
26     religion2 string = os.Getenv("LLM_TWO_RELIGION")
27
28     topic      string = os.Getenv("TOPIC")
29     turnsStr    string = os.Getenv("TURNS")
30     sentencesStr string = os.Getenv("SENTENCES")
31
32     // Global combined history of what the debaters say for the judge
33     // and debaters to read
34     transcript []string
35
36     // Output file
37     file *os.File
38 )

```

Figure 2.4: Global Variables (proj3.go)

As you can see, most of these variables are environment variables (Lines 20-30) that were specified in the `docker-compose.yml` file. There is also a global transcript slice that keeps track of what the debaters say (Line 33). This will eventually be passed to the judge for them to read and make a final decision. Finally, there is a file that will be created, and all output from this program will be written to this file (Line 36).

Below are some global constants that very important and used throughout the program.

```

39 // Declare constants
40 const (
41     // Debaters will only read the last MAX_TRANSCRIPT_LENGTH chats
42     // in the transcript to form their response
43     // The Judge will still read the entire transcript (unless too
44     // many tokens were passed in)
45     MAX_TRANSCRIPT_LENGTH = 4
46
47     // Constants for max sentences per turn and turn count
48     MAX_SENTENCES = 5
49     MAX_TURNS     = 8
50
51     // Constants for DEFAULT sentences per turn and turn count
52     DEFAULT_SENTENCES = 3
53     DEFAULT_TURNS     = 5
54 )

```

Figure 2.5: Global Constants (proj3.go)

The `MAX_TRANSCRIPT_LENGTH` specifies how much the debaters themselves will read in order to produce their response (Line 43). For example, since it is set to four, it will read its previous two answers and the opponents previous two answers (if it exists). The Judge still reads all responses, unless it reaches the token limit, which I will touch on a little later. There are also variables for the max sentence count and max turn count (Lines 46-47), as well as the default values for sentences and turns (Lines 50-51). These variables will be used if the user wrote some number that was too large OR if they wrote something that wasn't a number at all in the `docker-compose.yml` file for the environment variables.

Below are the self-made structures used in this program:

```

54 // Message structure that both request and response use
55 type ChatMessage struct {
56     Role    string 'json:"role"'
57     Content string 'json:"content"'
58 }
59
60 // Chat Request Format
61 type ChatRequest struct {
62     Model      string      'json:"model"'
63     Messages   [] ChatMessage 'json:"messages"'
64     Temperature float64    'json:"temperature"'
65     TopP       float64    'json:"top_p"'
66     Stop       [] string   'json:"stop"'
67 }
68
69 // Chat Response Format
70 type ChatResponse struct {
71     Choices []struct {
72         Message ChatMessage 'json:"message"'
73     } 'json:"choices"'
74     Usage struct {
75         PromptTokens    int 'json:"prompt_tokens"'
76         CompletionTokens int 'json:"completion_tokens"'
77         TotalTokens      int 'json:"total_tokens"'
78     } 'json:"usage"'
79 }

```

Figure 2.6: Structures (proj3.go)

These structures are based on generic chat API format, and many LLaMA backends (like llama.cpp or Ollama) expose an OpenAI-compatible API. That means they accept requests in exactly this structure and return responses in the same shape, like how its used with SmollLM2, even though we are using llama 3.2 as our default model [JSON Format for SmollLM2]. These structures are used to help organize the code when passing in request and receiving responses. The helper functions will help us interpret these values meaningfully.

In my opinion, the basis of this all is the `ChatMessage` structure, since this is

used in both the request and responses. It is used to request data from the LLM, and also used to receive data. The **Role** determines the importance of the prompt. In this program, the only roles used are the system and user roles, as will be explained later. The **Content** is the actual message.

The Chat Request will take in the model, messages, and other LLM parameters, like temperature, topP, and a stop slice. Temperature is a setting that controls randomness when picking words during text creation. “Top_p” or nucleus sampling is a setting that decides how many possible words to consider [[Understanding OpenAI Parameters23](#)].

For this project, the only fields used from the **ChatResponse** structure is the Choices and the Messages (and the Content in Messages). This is how it receives a response.

2.4 Helper Functions

Here I will explain the basis for all of the functions that help make this program work. Some of these functions are long, so I will only be explaining parts that I find worthy of an explanation.

```
check(e error)

86 // Ends program if there was an error
87 func check(e error) {
88     if e != nil {
89         log.Fatal(e)
90     }
91 }
92
93 // Wrapping text for writing to output file
```

Figure 2.7: check() (proj3.go)

This program will return an error if something went wrong. This function is called a lot throughout the program, especially in cases where if something goes wrong, it will mess up the entire program. This is not called when there are invalid environment variables (unless its the **BASE_URL** or **MODEL**), since the values can just be set to default ones.

writeToFile(text string)

I don’t feel the need to actual show the code for this function since it just does one specific thing, writes some **text** to a file. The part of this function that may seem complicated is the line wrapping functionality. If an entire response is written to the file, it will just write it all into one line, so part of this function is to wrap text to new lines if it reaches a certain character limit. This makes the output look a lot nicer.

```

printResponse(text string, flowed bool)

142 // Prints out response word by word
143 func printResponse(text string, flowed bool) {
144
145     // Write the text to the file
146     writeToFile(text)
147
148     // Just print the block of text if it doesn't need to be flowed
149     if !flowed {
150         fmt.Println(text)
151         return
152     }
153
154     // Split response into words
155     words := strings.SplitSeq(text, " ")
156
157     // If made it here, the text needs to flow word by word
158     // Print each word one by one
159     for word := range words {
160         fmt.Printf("%s ", word)
161         time.Sleep(50 * time.Millisecond)
162     }
163
164     // After the text is flowed out and fully printed, add a newline
165     fmt.Println()
166 }

```

Figure 2.8: printResponse() (proj3.go)

This function prints the `text` passed in. The `flowed` boolean determines whether the text will flow out word by word, like an actual LLM would. This is the reason why the `-it` flag is used when running this container since `docker-compose up` wouldn't actually flow out this output.

The program begins by writing this text to the file. Then, if the output does not need to flow out, it will just print it to the console and end. However, if it needs to flow, the text will be divided into words, and each word will print out every 50ms.

```

sendRequest(history []ChatMessage, removeNewLines bool)

168 // Sends a HTTP POST Request and receives a response from the LLM
169 func sendRequest(history []ChatMessage, removeNewLines bool) (
    string, bool) {
170
171     // Create the request
172     reqBody := ChatRequest{
173         Model:      model,
174         Messages:   history,
175         Temperature: 0.7,
176         TopP:       0.9,
177     }

```

```

179 // Marshal this data into bytes
180 reqBytes, err := json.Marshal(reqBody)
181 check(err)
182
183 // Create the HTTP POST Request
184 req, err := http.NewRequest("POST", BASE_URL+"chat/completions",
    bytes.NewBuffer(reqBytes))
185 check(err)
186
187 // Sets header for this request
188 req.Header.Set("Content-Type", "application/json")
189
190 // Client will do this request
191 client := &http.Client{}
192 resp, err := client.Do(req)
193 check(err)
194 defer resp.Body.Close()
195
196 // Check HTTP Status
197 if resp.StatusCode != http.StatusOK {
198     body, _ := io.ReadAll(resp.Body)
199
200     // If it was a 404 error, you must download the appropriate
    model
201     if resp.StatusCode == http.StatusNotFound {
202         log.Fatalf("Download the appropriate model by doing \"docker-
    compose up llm\": \n%s", body)
203     }
204
205     log.Printf("Llama API call failed! Status: %d, Body: %s\n",
    resp.StatusCode, body)
206     log.Println("Retrying with smaller transcript...")
207     log.Println()
208     return "err", true
209 }
210
211 // Get information from request into bytes
212 body, _ := io.ReadAll(resp.Body)
213
214 // Unmarshal the bytes into JSON format
215 var chatResp ChatResponse
216 err = json.Unmarshal(body, &chatResp)
217 check(err)
218
219 // Makes sure a response is returned (was an error)
220 if len(chatResp.Choices) == 0 {
221     return "(no response)", true
222 }
223
224 // Return the LLM's response
225 respText := chatResp.Choices[0].Message.Content

```

Figure 2.9: sendRequest() (proj3.go)

```

226
227 // Replace all new lines with just a space if it needs to be
    formatted
228 if removeNewLines {
229     respText = strings.ReplaceAll(respText, "\n", " ")
230 }
231
232 // Return this text and that it was successful (no error)
233 return respText, false
234 }

```

This function is how requests get sent to the LLM and how it receives a response. First of all, a new request is created using the message history sent in. The data gets converted to bytes and a HTTP POST request is created to the URL specified in the `docker-compose.yml`. A header is added to this request so it returns information in the JSON format we want.

The client will then perform this request. We will then check the HTTP Status. There are many different status codes that correspond to the HTTP GO library variable fields [[HTTP Status Codes](#)]. If the status returns a 404 error, that most likely means the LLM wasn't downloaded. If it receives another error, that may be due to the token amount, so the function ends and it probably will be retried. In this implementation of the project, the max sentences and max turns amount makes it so there is no chance of the token limit being reached, but JUST IN CASE it did, it is handled correctly and the program will not crash.

If the HTTP POST request was successful, the response is read and unmarshaled into JSON format. The content of this message will then be returned.

Sometimes, the messages may have a lot of newlines, which is hard to read when looking at the debate, so these can be removed and replaced with just spaces. This is what happens when a debater is making a request (all responses from debaters will have NO newlines). However, the judge response will contain new lines, because most of the time, they respond in a list of why one debater won over the other.

`judgeDebate()`

```

236 // judgeDebate runs the LLM0 (Judge) on the full debate history
237 func judgeDebate() {
238     printResponse("\n--- JUDGMENT PHASE: LLM0 (The Judge) ---", false
    )
239
240 // Improved system prompt for the judge
241 judgeSysPrompt := fmt.Sprintf(
242     "You are an impartial, non-biased, non-religious JUDGE. Read
    the debate between a %s follower (DEBATER_1) "+
243     "and a %s follower (DEBATER_2) on the topic: %s.",
244     religion1, religion2, topic,
245 )

```

```

247 // Send request to LLM
248 sendARequest := true
249 judgeResponse := ""
250
251 // Transcript may need to be shortened, so store it in another
    variable
252 currentTranscript := transcript
253
254 // Send the request until there is not an error
255 for sendARequest {
256
257     // Convert full debate into a string
258     fullDebate := strings.Join(currentTranscript, "\n")
259
260     // Prepare conversation for the judge
261     judgeHistory := []ChatMessage{
262         {Role: "system", Content: judgeSysPrompt},
263         {Role: "user", Content: "Here is the full debate:\n" +
264             fullDebate},
265         {Role: "user", Content: "YOU MUST CHOOSE THE WINNER AND
266             EXPLAIN WHY."},
267     }
268
269     // Send the request
270     resp, err := sendRequest(judgeHistory, false)
271
272     // If there was no error with the request, continue on
273     if !err {
274         judgeResponse = resp
275         sendARequest = false
276     } else if len(currentTranscript) > MAX_TRANSCRIPT_LENGTH {
277         // If there was an error (most likely max context size),
278         // resend the request with a shorter debate transcript
279         currentTranscript = currentTranscript[len(currentTranscript)-
280             MAX_TRANSCRIPT_LENGTH:]
281     } else {
282         // If the token limit is too large AND the transcription size
283         // is too small, SOMETHING WENT WRONG (THIS SHOULD NOT HAPPEN
284         // EVER)
285         judgeResponse = "Something went wrong when generating
286             response. Please try again later."
287         sendARequest = false
288     }
289 }
290
291 // Print its response
292 printResponse(judgeResponse, true)
293 }

```

Figure 2.10: judgeDebate() (proj3.go)

This function deals with the judge getting the transcript and processing which debater won. It starts off with the judge getting a system prompt telling it what to do. It then will send a request, passing it this system prompt, the full debate, and another prompt telling it to choose the winner and explain why.

The way the prompt is worded and set up makes it so the judge will decide a winner without a bias [[Prompt Formats for Llama 3.1](#)]. This request is sent to the `sendRequest()` function, and a response will be returned.

If there was an error with this request (and the program didn't prematurely end), that probably means the error has to do with the token limit. This means that the full debate will be cut a lot more so that the judge can respond (Again, with this current implementation with the max constants, the chances of this happening are VERY LOW, but it is handled nonetheless).

Then, this response is printed whenever a valid response comes through.

2.5 Main Program

Here I will be breaking down each part of main that I find worthy to explain, since a lot has already been explained in this Section.

```
299 // Create text file that will be the output of the debate
300 var err error
301 file, err = os.Create("/app/output/output.txt")
302 check(err)
303 defer file.Close()
```

Figure 2.11: `main()` Creating Output File (`proj3.go`)

The first significant part of this function is creating the output file, pointed to by the global `file` variable, so that the `printResponse()` function can use it. The OS will create the file in the `/app/output` directory, but remember from before that this directory is the same as the mounted volume path in `docker-compose.yml`, meaning that this file will also be put on the host directory. This file is created using the `os` function `Create`, and it will be closed when the program ends [[Writing Files25](#)].

```

305 // Make sure topic is valid
306 if topic == "" {
307     topic = "The War in Gaza"
308 }
309
310 // Assign given religions to LLM1 and LLM2
311 // If one of these variables were not set, or they were equal,
312 // use default religions
313 if religion1 == religion2 || religion1 == "" || religion2 == "" {
314     religion1 = "Muslim"
315     religion2 = "Jewish"
316 }
317
318 // Store how many turns each LLM has to speak (default is 5)
319 turns, err := strconv.Atoi(turnsStr)
320 if err != nil || turnsStr == "" || turns <= 0 {
321     fmt.Printf("Warning: Invalid turn amount: '%s'. Setting it to %d.\n", turnsStr, DEFAULT_TURNS)
322     turns = DEFAULT_TURNS
323 }
324 // The maximum turn count is 8
325 if turns > MAX_TURNS {
326     fmt.Printf("Warning: Too many turns for each LLM: %d. Setting turn count to %d.\n", turns, MAX_TURNS)
327     turns = MAX_TURNS
328 }
329
330 // Store how many sentences each LLM can to speak per turn (
331 // default is 3)
332 sentences, err := strconv.Atoi(sentencesStr)
333 if err != nil || sentencesStr == "" || sentences <= 0 {
334     fmt.Printf("Warning: Invalid sentence amount: '%s'. Setting it to %d.\n", sentencesStr, DEFAULT_SENTENCES)
335     sentences = DEFAULT_SENTENCES
336 }
337 // The maximum sentence count is 5
338 if sentences > MAX_SENTENCES {
339     fmt.Printf("Warning: Too many sentences for each LLM: %d. Setting sentence count to %d.\n", sentences, MAX_SENTENCES)
340     sentences = MAX_SENTENCES
341 }

```

Figure 2.12: main() Default Values (proj3.go)

This section of the function will validate all of the environment variables. First, any topic is a valid topic besides empty topics. If the topic variable was empty, it will set it to a default topic. Also, any religion is valid, except if they are empty or that they equal each other. If any of these cases are the same, the default religions will be set.

Then, the turn and sentence count will be set. Turns signifies how many turns each LLM gets to speak, and sentences signifies how much the LLM can speak per turn (rough estimate). If the topic/sentence count is NOT a number or it

is less than or equal to 0, it will be set to the default value. If it is a number, but it is too high, it will be set to the maximum value.

```
341 // Will store system messages for each LLM
342 sysMessages := map[int]ChatMessage{}
343
344 // Map religion to each LLM
345 religions := map[int]string{
346     1: religion1,
347     2: religion2,
348 }
349
350 // Set up system messages for both LLMs
351 for i := range 2 {
352     // Iteration 0: ID 1
353     // Iteration 1: ID 2
354     id := i + 1
355
356     // Set up initial system message for debater
357     sys := fmt.Sprintf('You are DEBATER_%d, a committed follower
358         and articulate representative of the %s worldview.
359
360         Your goal is to argue your faith's perspective on the
361         Topic: %s. Be respectful, persuasive, and concise.
362         You must argue ONLY from within the authentic
363         teachings, values, texts, and reasoning of the %s tradition.
364         Do not misrepresent or alter the faith. Do not invent
365         scripture.
366         Do not use the opponent's scripture or theology as
367         authoritative.
368
369         Every response you give **MUST** be %d sentences.',
370         id, religions[id], topic, religions[id], sentences)
371
372     // Add system message to the map
373     sysMessages[id] = ChatMessage{
374         Role:    "system",
375         Content: sys,
376     }
377 }
```

Figure 2.13: main() System Prompts for Debaters (proj3.go)

After these environment variables are verified and validated, each debater gets a system. **It is important to note that both debaters use the same LLM, but just different instances of it.** The LLMs are not in separate containers because they are pulled from the Docker Model Runner, as explained before. How these LLMs interact is like if they were completely separate. Each LLM is given their system prompt, signifying their religion and the topic that they will debate about. In this project, the first LLM is signified by id 1, and the second LLM is signified by id 2. This is how it corresponds to in the maps created for them.


```

384 // Start the debate
385 for turn := range turns * 2 {
386     // id 1 (Debater 1) speaks on even turns (0, 2, 4...)
387     // id 2 (Debater 2) speaks on odd turns (1, 3, 5...)
388     id := (turn % 2) + 1
389
390     // Determine the turn count (each LLM speaks in one turn)
391     turnCount := int(math.Floor(float64(turn/2)) + 1)
392
393     // Copy system message for this debater
394     history := append([]ChatMessage{}, sysMessages[id])
395
396     // Add new prompt
397     if len(transcript) > 0 {
398         history = append(history, ChatMessage{
399             Role: "user",
400             Content: fmt.Sprintf(
401                 'Respond to your opponent strictly from the perspective
of the %s worldview.
402                 Do NOT cite the opponent's scripture or use their
theology as authority.
403                 Use only arguments, principles, and reasoning that
someone fully committed to the %s tradition would genuinely use
.
404                 Do not quote scripture, doctrines, catechisms, hadith, or
commentaries unless you are absolutely certain they are real.
405                 If unsure, paraphrase the tradition's general viewpoint
rather than citing texts.',
406                 religions[id], religions[id],
407             ),
408         })
409
410     // Get the debate
411     currentFullDebate := strings.Join(transcript, "\n")
412
413     // Will only provide the last FOUR entries in the debate so
tokens will not overflow
414     if len(transcript) > MAX_TRANSCRIPT_LENGTH {
415         cutTranscript := transcript[len(transcript)-
MAX_TRANSCRIPT_LENGTH:]
416         currentFullDebate = strings.Join(cutTranscript, "\n")
417     }
418
419     // Let the LLM know the debate history up until this point
420     history = append(history, ChatMessage{
421         Role: "user",
422         Content: fmt.Sprintf("Provide a new counter-argument on the
topic %s with your own reasoning in %d sentence(s) "+
423         "(do not mention this sentence count, but you MUST adhere
to the rule). \n\nRemember, you are DEBATER_%d. "+
424         "Here is the full debate up until this point: \n%s",
425         topic, sentences, id, currentFullDebate),
426     })

```

```

427     } else {
428         // If there is no transcript yet, this means the LLM will
429         need to start the debate
430         history = append(history, ChatMessage{
431             Role: "user",
432             Content: fmt.Sprintf('Begin the debate. Present your
433             worldview's core position on the topic %s in %d sentence(s)
434             (do not mention this sentence count, but you MUST
435             adhere to the rule).
436             Speak exactly as a committed follower and scholar
437             of the %s tradition would.
438             Use real teachings, real values, and real
439             reasoning from that worldview.', topic, sentences, religions[id
440             ]),
441         })
442     }
443 }
444
445 // LLM responds
446 response, _ := sendRequest(history, true)
447
448 // Print real-time output
449 printResponse(fmt.Sprintf("\n-- DEBATER_%d: %s (TURN %d) --",
450 id, religions[id], turnCount), false)
451 printResponse(response, true)
452
453 // Add to the response to the transcript
454 transcript = append(transcript, fmt.Sprintf("DEBATER_%d: %s\n\n",
455 id, response))
456 }

```

Figure 2.14: main() The Debate (proj3.go)

This is the juicy part of the program; this is the debate. Since each LLM gets `turn` turns, the total amount of turns is `turns * 2`. Then, a history is formed for the debater. The history starts with the system prompt, then a new prompt is added (either telling the LLM to start the debate or to respond to their opponent). If there is a transcript, the LLM will argue with their opponent given the current transcript history (up to the last four debates). The LLM gets this transcript as a user prompt and will eventually interpret it and argue with their opponent over their points. If there is NO transcript yet, that means the debate hasn't started yet, so the LLM will start the debate. Because each LLM is given their own system prompt and their own user prompts, this is how **I created two different instances of the same LLM** without using containers and only using the Docker Model Runner to save space.

When the messages are created in the history, it will be sent to the `sendRequest` function to get a response. Then, this response is printed and added to the transcript. This all loops `turns * 2` times.

```
449 // The Judge will judge this debate since it is now finished
450 judgeDebate()
451
452 // Once the Judge has given their response, the program can end
453 fmt.Printf("\nProgram took %s to run.\n", time.Since(start))
```

Figure 2.15: main() Calling the Judge and Ending Program (proj3.go)

After the debate finishes (the for loop is complete), the judge will need to judge the debate. This function was explained earlier, but it will read that build transaction history to see what debater said what.

After this, the program will end.

3 Conclusion

3.1 Quick Notes

Before explaining my thoughts, I just wanted to give a couple of notes of why I did/did not do some things. I explained most of my thought process already, but I wanted to explain some other things.

At first, when thinking of how I could create this project, I wanted responses to be created concurrently with other ones being printed. This would speed up the project, since right now, after a previous response is printed, the program THEN calls the API and waits for a response. This seems slow, which it kind of is, but then I realized maybe that was a good thing. The reason why I kept it like this is for the user to read the debate easier. The text scrolls a little fast, and if the debate just kept going, it may be harder to read the whole thing. Having the debaters "pause" for a bit in between responses makes it easier to read it.

Currently, there is no actual way to "restart" a debate other than running the **docker run** command. In my implementation, I think this is a great way to run the program because running the same debate again would just be boring since it uses the same environment variables and parameters. To change the debate variables, you must go into the **docker-compose.yml** file (which is all neatly set up) and change those variables, build the image again, and run the container from the Image.

3.2 Use of AI

I am proud to say that I didn't use any AI to generate any of the code in this program. I spent the first couple of weeks studying to integrate LLMs with Docker without using external API keys, and I also researched what models I should use. However, I did use some AI in generating the system prompts and

user prompts for the LLMs and the Judge. I thought what knows better about LLMs than AI.

I did use some OpenAI ChatGPT Version 4. The main prompt I used was "Generate a system prompt for a debater that needs to debate about some topic given their religion". I tried to limit my use of AI, so I didn't necessarily the exact responses it gave me, but I used them as a reference to create my system and user prompts.

Other than that, I Googled everything else (as you can see by my references) to tried to learn everything myself to give myself a great learning experience.

3.3 My Thoughts

Overall this was a major learning experience for me, especially when dealing with cloud computing software principles. I learned how to write a program that integrates LLMs inside of Docker. It was an amazing, long, experience. Below are my references for this Project.

References

- | | |
|--|---|
| [Golang Build Images Guide ²⁴] | Docker Documentation. <i>Golang Build Images Guide</i> . 2024. URL: https://docs.docker.com/guides/golang/build-images . |
| [HTTP Status Codes] | IANA. <i>HTTP Status Codes</i> . URL: https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml . |
| [JSON Format for SmolLM2] | Docker Documentation. <i>JSON Format for SmolLM2</i> . URL: https://docs.docker.com/ai/model-runner/api-reference . |
| [Prompt Formats for Llama 3.1] | Llama Documentation. <i>Prompt Formats for Llama 3.1</i> . URL: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1 . |
| [Setting Up LLM Within Docker] | YouTube. <i>Setting Up LLM Within Docker</i> . URL: https://www.youtube.com/watch?v=G0gfQxDPaDw . |

- [Understanding OpenAI Parameters23] Miguel de la Vega. *Understanding OpenAI's "Temperature" and "Top_p" Parameters in Language Models*. 2023. URL: <https://medium.com/@1511425435311/understanding-openai-temperature-and-top-p-parameters-in-language-models-d2066504684f>.
- [UPX Source17] gruebel. *UPX Source*. 2017. URL: <https://github.com/gruebel/docker-upx>.
- [Using -it in docker run Command] Docker Documentation. *Using -it in docker run Command*. URL: <https://docs.docker.com/reference/cli/docker/container/run/#tty>.
- [Writing Files25] Mark McGranaghan and Eli Bendersky. *Go by Example: Writing Files*. 2025. URL: <https://gobyexample.com/writing-files>.